

Projeto de Disciplina – Etapa AV2
VidaPlena — Sistema de Gestão de Clínica Multidisciplinar
(Evolução)

Informações Gerais

- **Integrantes:** 4 à 7 membros
- **Data Entrega:** 28/06/2026
 - Para fins de avaliação, será considerada apenas a branch master/main do projeto (outras branches serão desconsideradas).
 - Para fins de avaliação, serão apenas os commits realizados até as 23:59 do dia 28/06/2026.
- **Forma de Entrega:** Projeto no github (professor incluído como participante – jonatasaraujo@gmail.com)

Entregáveis

1. **Código-fonte (0,7):**
 - a. projeto Java compilável e executável via terminal
 - b. necessidades de negócios totalmente desenvolvidas e executáveis
 - i. As descrições das necessidades de negócio estão disponibilizadas no documento “Projeto de Disciplina - Jornadas de Usuário.pdf”, na pasta “Projeto – Etapa 2”, na área de conteúdo da disciplina, no BB.
2. **Documentação (0,3):**
 - a. Diagrama de classes completo (mostrando hierarquias, interfaces, relacionamentos)
 - b. Descritivo do projeto (readme)
 - i. Descrição do sistema, contendo:
 1. Funcionalidades desenvolvidas
 2. Orientações para compilação e execução
 3. Descrição como realizar as operações presentes do sistema

Observações sobre avaliação

1. **O projeto deve ser originário a partir de um fork do projeto base, disponibilizado em: [LINK](#)**
2. **Projeto que não tenha origem a partir do projeto disponibilizado no link do item acima não será considerado entregue, logo não será considerado para fins de avaliação.**
3. Código que não compila terá nota máxima de 0,2 referente a componente de código-fonte.
4. **Cada membro do time deverá ter ao menos 3 commits de código funcional.**
5. Os commits devem ser incrementais e descritivos
 - a. Não será considerado 1 commit gigante com todo o código.
 - b. Todos os commit devem ter uma mensagem descrevendo o que está sendo adicionado/removido/modificado no código.
6. Uso de frameworks, bibliotecas externas ou ferramentas não vistas na disciplina implica perda de 0,1 ponto por ocorrência.
7. Caso seja detectado o uso de Inteligência Artificial em qualquer parte do projeto, a entrega será considerada nula e o grupo terá nota zero para este componente (projeto) da AV2.
8. O professor poderá solicitar defesa oral do grupo para validar a autoria do código.

Conceitos que DEVEM ser aplicados (itens obrigatórios)

1. Encapsulamento: todos os atributos devem ser privados com acesso via getters/setters.

2. Modificadores de acesso: visibilidade privada, protegida e pública devem ser usados com critério em classes, atributos e métodos.
3. Relacionamentos de Associação:
 - a. Exemplo: Consulta conhece Paciente e Profissional (referência direta ao objeto).
4. Relacionamentos de Agregação
Exemplo: Profissional possui lista de Horários disponíveis (horários existem independentemente)
5. Relacionamentos de Composição:
 - a. Exemplo: Atendimento contém Prontuário (prontuário não existe sem o atendimento)
6. Herança e hierarquia de classes:
7. Sobrecarga: manter construtores e métodos sobrecarregados já existentes e criar novos, conforme necessidade identificada.
8. Sobrescrita
9. Ligação dinâmica
10. Polimorfismo
 - a. Exemplos: Tratar “Pagamento” de forma polimórfica (PagamentoDinheiro, PagamentoCartao, PagamentoConvenio)
11. Dynamic casting:
 - a. Exemplo: Ao percorrer lista de “Pessoa”, fazer cast para “Paciente” ou “Profissional”, quando necessário
12. Classes abstratas
13. Interfaces
14. Interfaces e herança
15. Exceções (try/catch/finally/throws): tratamento em toda entrada de dados do usuário e operações críticas
16. Exceções personalizadas: para tratar situações relacionadas à indisponibilidade, inatividade, invalidez, item não encontrado, etc.
17. Coleções:
 - a. List: substituir arrays fixos por “ArrayList” para pacientes, profissionais, consultas, atendimentos
 - b. Set: usar “HashSet” para controlar CPFs cadastrados (evitar duplicação)
 - c. Map: usar “HashMap” para mapear CPF → Paciente e nome → Profissional (busca rápida)

Cenário de Negócio Expandido

A clínica VidaPlena cresceu. O que antes era um sistema simples com arrays fixos agora precisa suportar um volume maior de dados, novas regras de negócio e diferentes perfis de profissionais. A gerência decidiu investir na modernização do sistema, exigindo uma reestruturação completa do código.

O que mudou na clínica

1. Pessoas são a base de tudo

A gerência percebeu que Pacientes e Profissionais compartilham dados em comum: nome, CPF, telefone e data de nascimento. Para evitar duplicação de código, o sistema agora deve ter uma classe base “Pessoa”, que concentra esses atributos e comportamentos comuns.

Toda “Pessoa” deve ser capaz de “exibirResumo()”, mas cada tipo exibe informações diferentes — o paciente mostra dados de convênio e status, enquanto o profissional mostra especialidade, registro e valor da consulta.

O sistema agora mantém, além das listas separadas, uma lista unificada de todas as pessoas cadastradas. Quando o gerente pede um relatório geral de cadastros, o sistema percorre essa lista e chama “exibirResumo()”

em cada elemento — a ligação dinâmica garante que o comportamento correto seja executado. Em alguns relatórios, é necessário saber se a pessoa é paciente ou profissional para exibir informações extras.

2. Profissionais especializados

Cada especialidade agora tem comportamentos próprios:

- “Fisioterapeuta”: pode registrar **sessões** (quantidade de sessões previstas no plano de tratamento). Possui atributo “totalSessoesPrevistas”.
- “Psicólogo”: pode registrar **abordagem terapêutica** (ex: TCC, psicanálise, humanista). Possui atributo “abordagem”.
- “Nutricionista”: pode registrar **plano alimentar** (texto descritivo). Possui atributo “planoAlimentar”.
- “ClinicoGeral”: pode registrar **encaminhamentos** para outras especialidades. Possui atributo “encaminhamento”.

Todos têm por base a entidade “Profissional”, que por sua vez deriva de “Pessoa”. Cada um possui o seu próprio comportamento “exibirResumo()” para incluir suas informações específicas. Também sobrescrevem um método “registrarEspecifico(Atendimento atendimento)” que adiciona ao atendimento as informações particulares da especialidade.

3. Prontuário como composição

Cada atendimento agora **contém** um “Prontuario”. O prontuário não existe sem o atendimento — se o atendimento for removido, o prontuário também é. O prontuário armazena: observações, diagnóstico, procedimentos realizados (“List<String>”) e a data do registro. Essa é uma relação de **composição**.

4. Horários como agregação

Os horários disponíveis de um profissional agora são objetos “HorarioDisponivel” (com dia da semana e turno: manhã/tarde). Um mesmo horário pode ser reutilizado se o profissional mudar de agenda — os horários existem independentemente do profissional. Essa é uma relação de **agregação**.

5. Formas de pagamento polimórficas

O pagamento deixa de ser uma forma única. Agora existe uma entidade chamada “Pagamento” com o comportamento comum “calcularValorFinal()”, que deve ser construída por todas as formas de pagamento que derivem dela. Neste novo momento, a clínica possui as seguintes formas de pagamento:

- Pagamento em dinheiro (ou pix): aplica 5% de desconto sobre o valor.
- Pagamento com cartão: permite parcelamento em até 6x; acima de 3x, aplica taxa de 2,5% por parcela extra.
- Pagamento por convênio: aplica cobertura percentual do convênio (varia por convênio: SaúdePlus = 40%, VidaMais = 30%, BemEstar = 50%).

O sistema trata todos como “Pagamento” (polimorfismo), mas cada um calcula o valor final de forma diferente.

6. Consulta com múltiplos contratos

A classe “Consulta” agora deve construir as seguintes operações, oriundas de diferentes entidades:

- “Agendavel”: define os métodos: “agendar()”, “cancelar()”, “remarcar()”
- “Exportavel”: define o método: “exportarDados()”, que retorna uma String formatada com todos os dados da consulta (para futura integração com outros sistemas)

Isso demonstra que uma classe pode assumir múltiplos "contratos" simultaneamente.

O contrato existente em “Exportavel” também é implementado por “Atendimento” e “Pagamento”, permitindo que o sistema exporte dados de qualquer entidade de forma uniforme.

7. Busca rápida com Maps e controle com Sets

O sistema usa:

- um “HashMap<String, Paciente>” para buscar pacientes por CPF.
- um “HashMap<String, Profissional>” para buscar profissionais por nome.
- um “HashSet<String>” para garantir unicidade de CPFs antes de inserir.

As consultas, atendimentos e pagamentos são armazenados em uma lista editável.

8. Convênios como objetos

O convênio deixa de ser apenas um nome. Agora é uma entidade com propriedades: nome, percentual de cobertura e lista de especialidades cobertas. O paciente possui uma **associação** com “Convenio” (pode ter ou não; o convênio existe independentemente do paciente).

9. Exceções e Tratamento de Erros

A versão anterior do sistema não tratava erros — qualquer entrada inválida causava uma falha silenciosa ou travamento. Na nova versão, **toda operação crítica deve ser protegida** e o sistema nunca deve encerrar abruptamente.

Para isso, devem ser criadas formas personalizadas para tratar diferentes tipos de erros que possam ocorrer. O sistema deve definir as seguintes exceções:

1. PacienteInativoException: tentativa de agendar consulta para paciente inativo.
2. PacienteNaoEncontradoException: CPF informado não existe no sistema.
3. ProfissionalNaoEncontradoException: nome do profissional não existe no sistema.
4. HorarioIndisponivelException: o profissional não atende no dia ou horário já está ocupado.
5. ConsultaNaoEncontradaException: tentativa de operar sobre consulta inexistente.
6. OperacaoInvalidaException: tentativa de cancelar consulta já realizada, registrar atendimento em consulta não agendada, etc.
7. PagamentoInvalidoException: valor negativo, tipo de pagamento não reconhecido, parcelas fora do limite.
8. ConvenioNaoCobreException: especialidade da consulta não é coberta pelo convênio do paciente |

Cada exceção deve ter:

- Uma mensagem descritiva passada no construtor.
- Sobrecarga de construtor: um que recebe apenas a mensagem e outro que recebe mensagem + causa.

10. Onde o tratamento deve aparecer

Entrada de dados do usuário (try/catch/finally):

Toda leitura de dados numéricos (idade, valor, índice, parcelas) deve estar dentro de “try/catch” para capturar “NumberFormatException”. Quando o usuário digita algo inválido, o sistema exibe uma mensagem amigável e solicita novamente, sem travar.

O bloco “finally” deve ser usado para garantir que recursos sejam liberados ou mensagens de log sejam exibidas. Exemplo: ao final de qualquer operação de pagamento, o bloco “finally” exibe “--- Operação de pagamento finalizada ---” independentemente de sucesso ou falha.

Operações de negócio (throws + tratamento no chamador):

Os métodos de serviço devem **declarar** as exceções que podem lançar.

O chamador (menu/Main) captura cada exceção e exibe a mensagem adequada ao usuário.

11. Cenários obrigatórios de exceção no fluxo do sistema

O sistema deve tratar corretamente (sem travar) **todos** os seguintes cenários:

1. Usuário digita texto onde se espera número (ex: "abc" no campo idade)
2. Tentativa de agendar para paciente inativo → "PacienteInativoException"
3. Tentativa de agendar com CPF inexistente → "PacienteNaoEncontradoException"
4. Tentativa de agendar com profissional inexistente → "ProfissionalNaoEncontradoException"
5. Tentativa de agendar em horário já ocupado → "HorarioIndisponivelException"
6. Tentativa de agendar em dia que o profissional não atende → "HorarioIndisponivelException"
7. Tentativa de cancelar consulta já realizada → "OperacaoInvalidaException"
8. Tentativa de cancelar consulta já cancelada → "OperacaoInvalidaException"
9. Tentativa de registrar atendimento em consulta não agendada → "OperacaoInvalidaException"
10. Tentativa de pagamento com tipo inválido (ex: "cheque/voucher") → "PagamentoInvalidoException"
11. Tentativa de parcelar em mais de 6x ou menos de 1x → "PagamentoInvalidoException"
12. Tentativa de pagamento via convênio quando a especialidade não é coberta → "ConvenioNaoCobreException"
13. Busca de consulta por CPF + data + horário sem resultado → "ConsultaNaoEncontradaException"

Requisitos Obrigatórios para Fixação de Conceitos

Os itens abaixo são **obrigatórios** e têm finalidade pedagógica. Mesmo que existam formas mais simples de resolver o problema, o aluno **deve** implementar conforme descrito para demonstrar domínio do conceito.

R1 — Encapsulamento (nenhum atributo público)

- TODOS os atributos de TODAS as classes devem ser privados (ou protegidos, quando justificado pela herança).
- Criar getters e setters para cada atributo que precise ser acessado externamente.
- Ao menos 2 setters devem conter **validação interna**
 - exemplos:
 - "setIdade(int idade)" recusa valores negativos;
 - "setCpf(String cpf)" recusa string vazia.
- **Justificativa pedagógica:** encapsulamento não é apenas "colocar private e gerar getter/setter", mas sim **proteger o estado interno** do objeto com regras.

R2 — Modificadores de acesso com propósito

- A classe "Pessoa" pode ter atributos protegidos (exceção, quando justificável) para que subclasses acessem diretamente, mas classes externas não.
- Métodos auxiliares internos (ex: cálculos intermediários) devem ser privados.
- Ao menos 1 método deve ser protegido em "Profissional" (acessível apenas pelas especializações).
- Não usar visibilidade pública em tudo por conveniência. Será verificado se há critério na escolha.
- **Justificativa pedagógica:** demonstrar que sabe **por que** escolheu cada modificador, não apenas qual usar.

R3 — Herança com profundidade mínima de 3 níveis

- A hierarquia deve ter no mínimo 3 níveis

- Exemplo: “Pessoa” → “Profissional” → “Fisioterapeuta”.
- Cada nível deve adicionar ao menos 1 atributo e 1 método próprio.
- Construtores das subclasses devem chamar o construtor da superclasse explicitamente.
- Demonstrar que um “Fisioterapeuta” **é um** “Profissional” que **é uma** “Pessoa”.
- **Justificativa pedagógica:** hierarquia rasa (2 níveis) não exercita suficientemente o conceito de cadeia de herança e propagação de construtores.

R4 — Sobrecarga vs. Sobrescrita (distinção clara)

- Manter ao menos **4 classes com construtores sobrecarregados** (já existentes da AV1).
- Manter ao menos **4 classes com métodos sobrecarregados** (mesmo nome, parâmetros diferentes).
- Usar “@Override” em todo método sobrescrito. O código **não deve compilar** se a anotação for removida e a assinatura estiver errada.
- Em comentário no código, o aluno deve indicar ao menos 1 exemplo de cada com a anotação:
 - Exemplo:

// SOBRECARGA: mesmo nome, parâmetros diferentes (resolvido em tempo de compilação)

// SOBRESCRITA: mesmo nome e parâmetros, classe filha redefine comportamento (resolvido em tempo de execução)

- **Justificativa pedagógica:** a confusão entre sobrecarga e sobrescrita é o erro conceitual mais comum. Forçar a anotação e o comentário obriga a reflexão.

R5 — Ligação dinâmica (demonstração explícita)

- Criar um trecho de código onde uma “List<Pessoa>” é percorrida e “exibirResumo()” é chamado em cada elemento.
- O resultado deve ser **diferente** para Paciente e para cada tipo de Profissional, demonstrando que o método executado é decidido em tempo de execução.
- Criar um trecho onde uma “List<Pagamento>” é percorrida e “calcularValorFinal()” é chamado — cada subclasse retorna valor diferente.
- Em comentário no código:

// LIGAÇÃO DINÂMICA: o método chamado depende do tipo REAL do objeto, não do tipo da referência

- **Justificativa pedagógica:** observar na prática que “Pessoa p = new Fisioterapeuta(...)” chama o “exibirResumo()” do Fisioterapeuta, não o de Pessoa.

R6 — Classes abstratas

- Pessoa e Pagamento devem ser abstratas.
- Cada classe abstrata deve ter ao menos **1 método abstrato** e ao menos **1 método concreto**.
- **Justificativa pedagógica:** entender a diferença entre “classe que não pode ser instanciada” e “classe que não tem nenhuma implementação” (interface).

R7 — Interfaces e herança múltipla de tipos

- Criar ao menos **2 interfaces**.
- Ao menos **1 classe** deve implementar ao menos **2 interfaces** simultaneamente
 - Exemplo: “Consulta implements Agendavel, Exportavel”
- Ao menos **3 classes diferentes** devem implementar a mesma interface .
- Demonstrar uso polimórfico via interface.
- **Justificativa pedagógica:** entender a diferença entre herança de implementação (extends, única) e herança de tipo (implements, múltipla).

R8 — Relacionamentos com semântica correta

- Implementar ao menos **1 exemplo de cada** relacionamento:
 - **Associação, Agregação, Composição**
- Em comentário no código, justificar a escolha:
// ASSOCIAÇÃO: Paciente conhece Convenio, mas ambos existem independentemente
// AGREGAÇÃO: Profissional possui horários, mas horários sobrevivem sem o profissional
// COMPOSIÇÃO: Prontuário só existe dentro de Atendimento — se Atendimento for removido, Prontuário também é
- Para **composição**: O construtor de “Prontuario” deve ser **package-private** ou o Prontuario deve ser criado **apenas** dentro de Atendimento (nunca externamente).
- **Justificativa pedagógica**: a maioria dos alunos confunde os três tipos. Forçar a implementação com semântica correta e comentário explicativo fixa a diferença.

R9 — Exceções com propósito (não apenas try/catch genérico)

- Criar ao menos **5 exceções personalizadas**.
- Cada exceção deve ter ao menos **2 construtores** (sobrecarga): um com mensagem e outro com mensagem + causa.
- Métodos de serviço devem declarar “throws” com as exceções específicas (não usar “throws Exception” genérico).
- O chamador deve ter blocos “catch” **separados** para cada exceção (não usar apenas “catch (Exception e)”).
- Usar “finally” em ao menos **2 operações** distintas com propósito real (mensagem de log, liberação de recurso simulada, etc.).
- **Proibido**: Usar “catch” vazio (“catch (Exception e) {}”). Todo catch deve ao menos exibir a mensagem.
- **Justificativa pedagógica**: entender a diferença entre exceções verificadas e não verificadas, e porque exceções específicas são melhores que genéricas.

R10 — Coleções com justificativa de escolha

- Substituir **todos** os arrays fixos + contadores por coleções do Java Collections.
- Usar ao menos **1 List, 1 Set e 1 Map**.
- Em comentário, justificar por que cada estrutura foi escolhida:
// ArrayList<Consulta>: ordem de inserção importa; acesso por índice necessário
// HashSet<String>: apenas verificação de existência (contains); não precisa de ordem
// HashMap<String, Paciente>: busca por chave (CPF); mais eficiente que percorrer lista
- Demonstrar ao menos **1 operação** característica de cada:
 - List: add(), get(index), iteração com for-each
 - Set: add(), contains(), tentativa de adicionar duplicata (retorna false)
 - Map: put(), get(key), containsKey(), iteração sobre “values()” ou “entrySet()”
- **Proibido**: Usar array de tamanho fixo para as entidades principais (pacientes, profissionais, consultas, atendimentos, pagamentos).
- **Justificativa pedagógica**: escolher a estrutura de dados adequada ao problema, não apenas “usar ArrayList para tudo”.

Restrições Técnicas

- **Linguagem**: Java (JDK 11 ou superior)
- **Entrada/Saída**: Scanner para entrada; System.out para saída (console)

- **Proibido:** frameworks (Spring, etc.), bibliotecas externas, banco de dados, interface gráfica.
- **Permitido:** apenas classes do pacote `java.util` (Collections Framework, Scanner) e `java.lang`.